

Sistema Inteligente de Contagem de Parafusos para Picking Industrial

Relatório Técnico – Desafio 1
Bootcamp de Ciência de Dados e Inteligência Artificial (CDIA)
Programa de Residência em IA

Autor: Ana Rafaely

Data: 30 de maio de 2026

Duas soluções desenvolvidas e comparadas

- | | |
|------------------|---|
| Solução 1 | Visão Clássica com OpenCV
(pipeline de segmentação adaptativa) |
| Solução 2 | Aprendizado Profundo com YOLOv11
(detecção supervisionada) |

0 Sumário

1	Introdução e Contexto do Problema	3
1.1	Objetivos	3
1.2	Restrições	3
1.3	Estratégia adotada	3
2	Solução 1 – Visão Clássica com OpenCV	3
2.1	Motivação	3
2.2	Arquitetura do Pipeline	4
2.2.1	Camada 1 – Análise de Cena	4
2.2.2	Camada 2 – Segmentação Adaptativa (6 Modos)	4
2.2.3	Camada 3 – Detectores	5
2.2.4	Camada 4 – Ensemble e Seleção	5
2.3	Estimação de Área de Referência	6
2.4	Modulo de Calibração	6
2.5	Avaliação e Métricas	6
2.6	Qualidade de Captura	6
2.7	Resultados Práticos – Testes com Novas Imagens	7
2.8	Grade de Debug – Pipeline Interno	7
2.9	Métricas de Avaliação – Visão Clássica	8
2.10	Grid de Resultados Consolidado	9
3	Solução 2 – Detecção com YOLOv11	10
3.1	Motivação	10
3.2	Dataset	11
3.3	Treinamento	11
3.4	Módulo de Inferência	12
3.5	Hiperparâmetros de Inferência	12
3.6	Exportação Mobile	12
3.7	Métricas do Treinamento – Curvas e Evolução	13
3.8	Matriz de Confusão	13
3.9	Curvas de Precisão, Recall e F1	14
3.10	Lotes de Validação	15
3.11	Resultados Práticos – Testes com Novas Imagens	15
4	Aplicação Web Compartilhada	16
4.1	Por que FastAPI?	16
4.2	Fluxo de uma requisição	16
4.3	Rotas Principais	17
4.4	Banco de Dados – Modelo ORM	17
4.5	Funcionalidades Exclusivas da Solução 1 (Web)	17
4.6	Conexão via Smartphone	18
5	Processo de Design – UX/UI da Aplicação	18
5.1	Contexto e Objetivos do Design	18
5.2	Protótipo de Interface – UX/UI	18
5.3	Decisões de Design Justificadas	20
5.4	Fluxo de Navegação	20

5.5	Responsividade e Compatibilidade Mobile	20
6	Análise Comparativa das Soluções	21
6.1	Comparação Técnica	21
6.2	Cenários Favoráveis a Cada Solução	22
7	Veredicto: Qual Solução é Melhor?	22
8	Critérios de Avaliação – Resposta Direta	22
8.1	1. Organização e Estruturação do Código	22
8.2	2. Criatividade na Proposta	23
8.3	3. Clareza nos Critérios de Tomada de Decisão	23
8.4	4. Capacidade Analítica	23
8.5	Bônus: Solução em Aplicação Web	24
8.6	Bônus: Baixo Recurso Computacional	24
9	Instruções de Execução	24
9.1	Solução 1 – Visão Clássica	24
9.2	Solução 2 – YOLOv11	24
10	Conclusão	24
A	Estrutura de Arquivos	25
B	Dependências Principais	26

1 Introdução e Contexto do Problema

Uma empresa metal-mecânica solicitou ao Programa de Residência em IA o desenvolvimento de um sistema capaz de automatizar a contagem de parafusos no processo de *picking*: a coleta e embalagem de itens que serão enviados a clientes. Atualmente, a contagem é realizada manualmente por funcionários que registram a quantidade em um *smartphone* corporativo, processo suscetível a erros humanos decorrentes de fadiga e alto volume de pedidos.

1.1 Objetivos

1. Desenvolver um sistema de visão computacional capaz de contar parafusos em fotografias tiradas por smartphones.
2. Implementar a solução em Python.
3. Preparar relatório técnico com metodologia, modelos e resultados.

1.2 Restrições

- A solução deve operar nos smartphones já disponíveis (baixo recurso computacional).
- A interface deve ser simples para operadores sem treinamento técnico.
- Apenas **5 imagens** foram fornecidas inicialmente, exigindo criatividade na aquisição ou augmentação de dados.

1.3 Estratégia adotada

Diante da complexidade do problema e das restrições, foram desenvolvidas **duas soluções independentes** e uma **aplicação web compartilhada**, permitindo comparação direta de abordagens:

- **Solução 1:** pipeline clássico de visão computacional com OpenCV – sem treinamento, interpretável, leve.
- **Solução 2:** detecção supervisionada com YOLOv11 – treinada em dataset público anotado, robusta e precisa.

2 Solução 1 – Visão Clássica com OpenCV

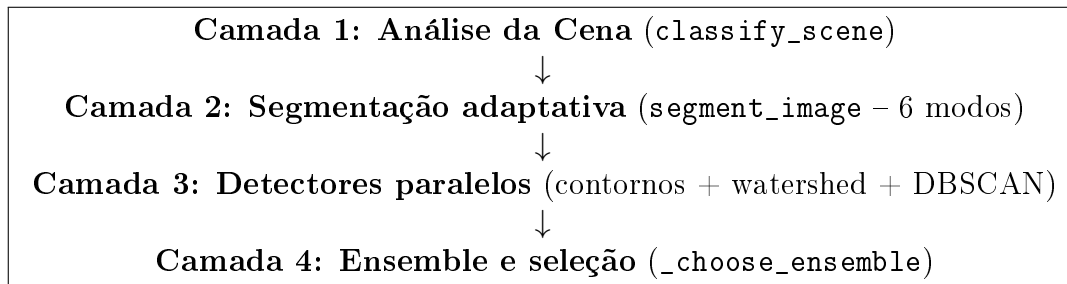
2.1 Motivação

Uma abordagem clássica foi escolhida como *baseline* por ser:

- **Zero-shot:** não requer dados anotados nem treinamento.
- **Interpretável:** cada etapa pode ser inspecionada visualmente.
- **Extremamente leve:** roda em qualquer dispositivo com OpenCV.
- **Rápida de implantar:** basta instalar as dependências e executar.

2.2 Arquitetura do Pipeline

O módulo central é `screw_counter.py`, com mais de 1.000 linhas que implementam um pipeline modular em quatro camadas:



2.2.1 Camada 1 – Análise de Cena

Antes de qualquer segmentação, a função `classify_scene` calcula indicadores visuais que caracterizam a cena e recomendam o modo mais adequado:

```

1 def classify_scene(img_bgr: np.ndarray) -> dict[str, Any]:
2     gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
3     mean_brightness = float(np.mean(gray))
4     contrast = float(np.std(gray))
5     light_bg_ratio = float(np.mean(gray > 185))
6
7     lap = cv2.Laplacian(gray, cv2.CV_32F)
8     noise_estimate = float(np.std(lap))
9     texture_ratio = float(np.mean(np.abs(lap) > max(12.0,
10     noise_estimate * 0.45)))
11
12     # Calcula densidade e toque entre objetos
13     if touching_score > 0.25 or density > 0.15:
14         mode = "modo_denso"
15     elif scale_variation > 3.0:
16         mode = "modo_multiescala"
17     elif tray_like:
18         mode = "modo_bandeja"
19     elif textured:
20         mode = "modo_texturizado"
21     elif light_bg_ratio > 0.55:
22         mode = "modo_claro"
23     else:
24         mode = "modo_cinza"
25     return {..., "recommended_mode": mode}
  
```

Listing 1: Análise de cena para escolha automática de modo

2.2.2 Camada 2 – Segmentação Adaptativa (6 Modos)

Cada modo usa uma combinação específica de filtros e limiares. A Tabela 1 resume as estratégias.

Tabela 1: Modos de segmentação e suas estratégias

Modo	Estratégia principal
modo_claro	Otsu invertido + Canny; fundo branco/claro
modo_cinza	Gradiente morfológico + Otsu; fundo cinza
modo_bandeja	CLAHE no canal L* (LAB) + gradiente; fundos coloridos
modo_texturizado	Top-hat para remover fundo + gradiente; fundos com textura
modo_denso	Bilateral + limiar adaptativo + Otsu combinados; objetos agrupados
modo_multiescala	Gradiente em três escalas (0.75×, 1.0×, 1.25×)

2.2.3 Camada 3 – Detectores

Três detectores são aplicados em paralelo sobre a mesma máscara:

detector_contours:

extrai componentes conectados e estima multiplicidade por razão de área ($\text{mult} = \lfloor A_c/A_{\text{ref}} \rfloor$ quando $A_c > 1.75 \cdot A_{\text{ref}}$).

detector_watershed:

aplica *distance transform* e localiza picos (máximos locais) para separar parafusos sobrepostos.

detector_fragmentos_dbscan:

agrupa fragmentos pequenos por DBSCAN (quando muitos componentes são menores que $0.7 \cdot A_{\text{ref}}$), contando cada *cluster* como um parafuso.

```

1 def _distance_peaks(mask, ref_area, min_distance=None):
2     dist = cv2.distanceTransform(mask_bin, cv2.DIST_L2, 5)
3     if min_distance is None:
4         min_distance = max(int(np.sqrt(ref_area / np.pi) * 0.62),
5                             5)
6     smooth = cv2.GaussianBlur(dist, (0, 0), sigmaX=min_distance /
7                                   2.5)
8     dilated = cv2.dilate(smooth, _kernel(2 * min_distance + 1))
9     peak_mask[(smooth == dilated) & (smooth > smooth.max() * 0.28)]
10    = 255
11    n_peaks, _, _, _ = cv2.connectedComponentsWithStats(peak_mask,
12                                                         8)
13    return dist, peak_mask, n_peaks - 1

```

Listing 2: Watershed: picos do distance transform como separadores

2.2.4 Camada 4 – Ensemble e Seleção

Com o modo `ensemble=True`, a função `_choose_ensemble` combina os resultados de todos os modos e detectores (até $6 \times 3 + 1 = 19$ candidatos), selecionando o mais estable por um score ponderado:

$$\text{score} = 1.2 \cdot \overline{\text{conf}} + \frac{1}{1 + |c - \text{mediana}|} - 0.05 \cdot \text{rank}$$

onde $\overline{\text{conf}}$ é a confiança média das detecções e rank penaliza detectores menos adequados para a cena atual.

2.3 Estimação de Área de Referência

A área de um parafuso individual (`ref_area`) é estimada automaticamente:

```

1 def estimate_ref_area(components, fallback=4021.0):
2     areas = sorted(float(c.area) for c in components)
3     if len(areas) >= 4:
4         # Remove 20% dos extremos (outliers) antes da mediana
5         trimmed = areas[int(len(areas)*0.20) : int(len(areas)*0.80)]
6         ref = float(np.median(trimmed or areas))
7     else:
8         ref = float(np.median(areas))
9     return float(np.clip(ref, fallback * 0.45, fallback * 2.5))

```

Listing 3: Estimação robusta da área de referência

2.4 Modulo de Calibração

A função `calibrate_from_single_screw` permite que o operador fotografe um único parafuso centralizado para calibrar a `ref_area` antes do uso. O sistema retorna confiança (*alta/média/baixa*) e orientações ao usuário.

2.5 Avaliação e Métricas

O módulo `metrics.py` define as métricas de avaliação: **MAE** (erro absoluto médio), **acurácia exata**, **viés médio**, supercontagens e subcontagens. O script `evaluation.py` executa avaliação em lote, salva grades de debug e gera relatório Markdown automático.

```

1 def mean_absolute_error(y_true, y_pred):
2     """Media dos erros absolutos (MAE)."""
3     return float(np.mean(np.abs(np.asarray(y_true) - np.asarray(
4         y_pred))))
5
6 def exact_match_accuracy(y_true, y_pred):
7     """Proporcao de contagens exatamente corretas."""
8     return float(np.mean(np.asarray(y_true) == np.asarray(y_pred)))
9
10 def mean_bias_error(y_true, y_pred):
11     """Vies medio: positivo = supercontagem, negativo = subcontagem"""
12     return float(np.mean(np.asarray(y_pred) - np.asarray(y_true)))

```

Listing 4: Métricas de avaliação (`metrics.py`)

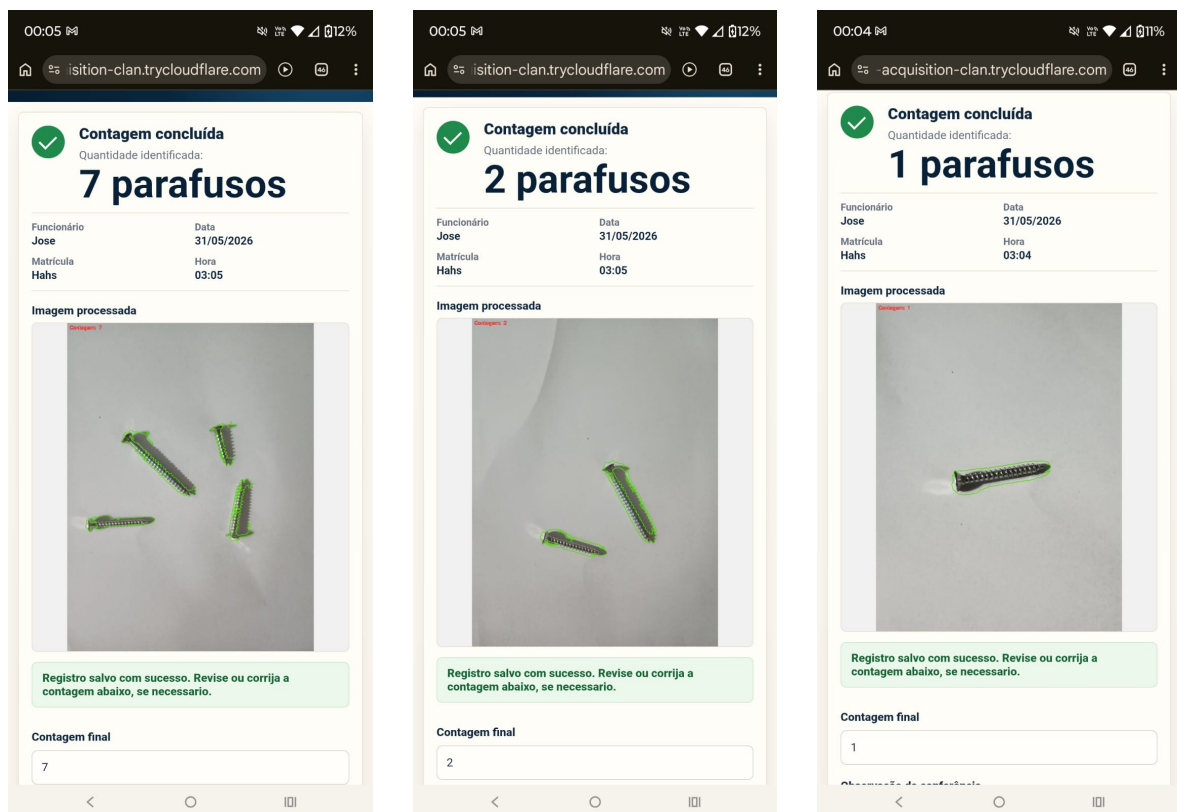
2.6 Qualidade de Captura

A função `_estimate_capture_quality` analisa a imagem e emite alertas ao operador: foto tremida (score de Laplaciano), objetos muito pequenos (distância), parafusos cortados na borda, e baixa confiança do ensemble.

- Sensível a variações de iluminação e fundo.
- Parafusos fortemente sobrepostos podem ser contados como um único blob.
- Fundos muito texturizados confundem o segmentador.
- Não generaliza para novos tipos de parafusos sem reajuste de parâmetros.

2.7 Resultados Práticos – Testes com Novas Imagens

Para validar o pipeline clássico em condições reais, o sistema foi testado com novas imagens fornecidas fora do conjunto de calibração. A Figura 1 apresenta três resultados representativos, onde cada parafuso detectado recebe uma caixa delimitadora colorida de acordo com o detector que o identificou (**verde** = contorno isolado, **laranja** = watershed, **vermelho** = DBSCAN).



(a) Resultado OpenCV – imagem 1

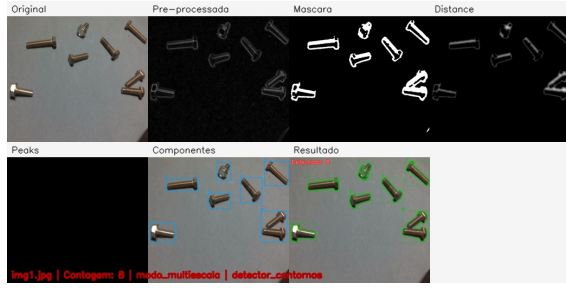
(b) Resultado OpenCV – imagem 2

(c) Resultado OpenCV – imagem 3

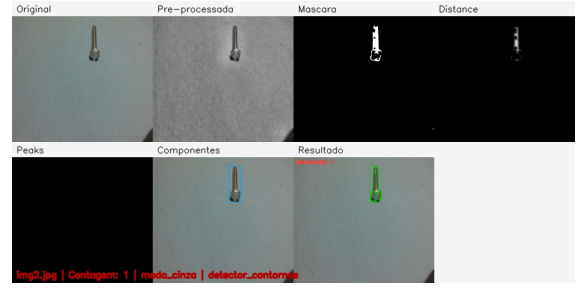
Figura 1: Resultados práticos do pipeline clássico em novas imagens.

2.8 Grade de Debug – Pipeline Interno

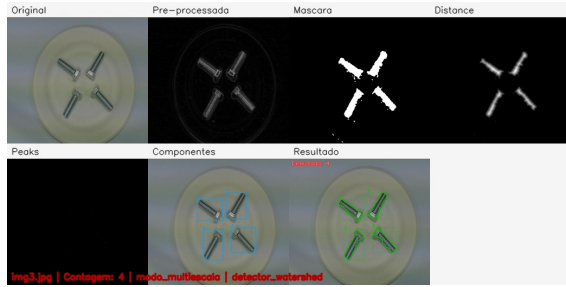
A Figura 2 exibe a grade de debug gerada pela função `make_debug_grid`, mostrando os estágios internos do pipeline: imagem original, pré-processada (CLAHE), máscara binária, *distance transform*, mapa de picos (watershed) e imagem anotada final. Esta visualização é fundamental para auditar e depurar casos-limite.



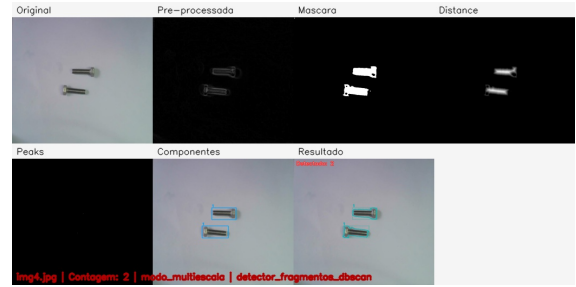
(a) Debug – imagem 1



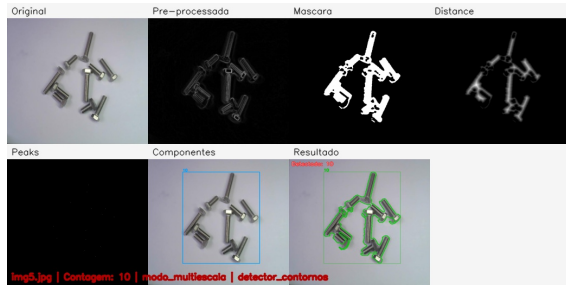
(b) Debug – imagem 2



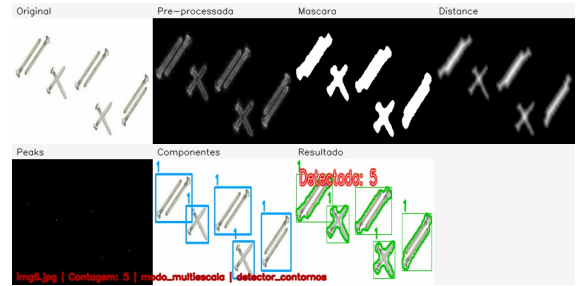
(c) Debug – imagem 3



(d) Debug – imagem 4



(e) Debug – imagem 5



(f) Debug – imagem 6

Figura 2: Grades de debug do pipeline clássico: cada painel exibe os estágios de pré-processamento, segmentação, *distance transform* e resultado.

2.9 Métricas de Avaliação – Visão Clássica

As Figuras 3 e 4 apresentam, respectivamente, o painel de avaliação quantitativa (MAE, acurácia exata e viés por imagem) e a análise de discriminação de *features* usadas pelo classificador de cena.

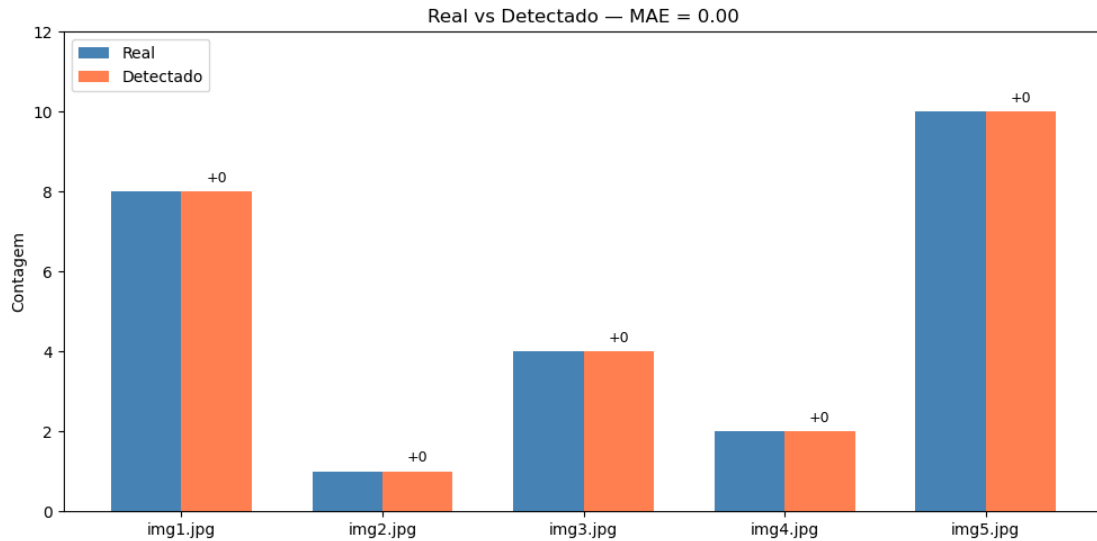


Figura 3: Painel de avaliação quantitativa da Solução 1: MAE, acurácia exata e erros assinados por imagem.

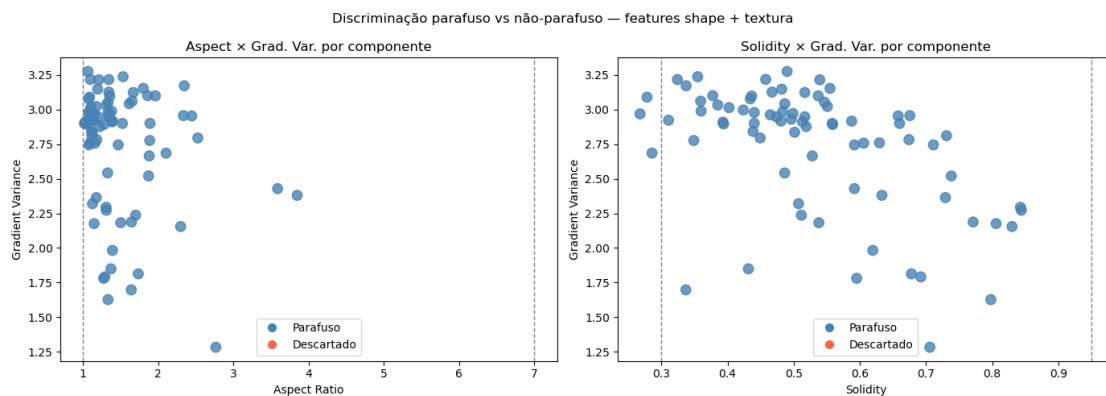


Figura 4: Análise das *features* de discriminação de cena usadas pelo `classify_scene`: brilho, contraste, textura e densidade.

2.10 Grid de Resultados Consolidado

A Figura 5 apresenta o painel consolidado com todos os resultados de contagem do pipeline clássico sobre o conjunto de imagens avaliado, permitindo visualizar de forma comparativa o desempenho em diferentes cenários.

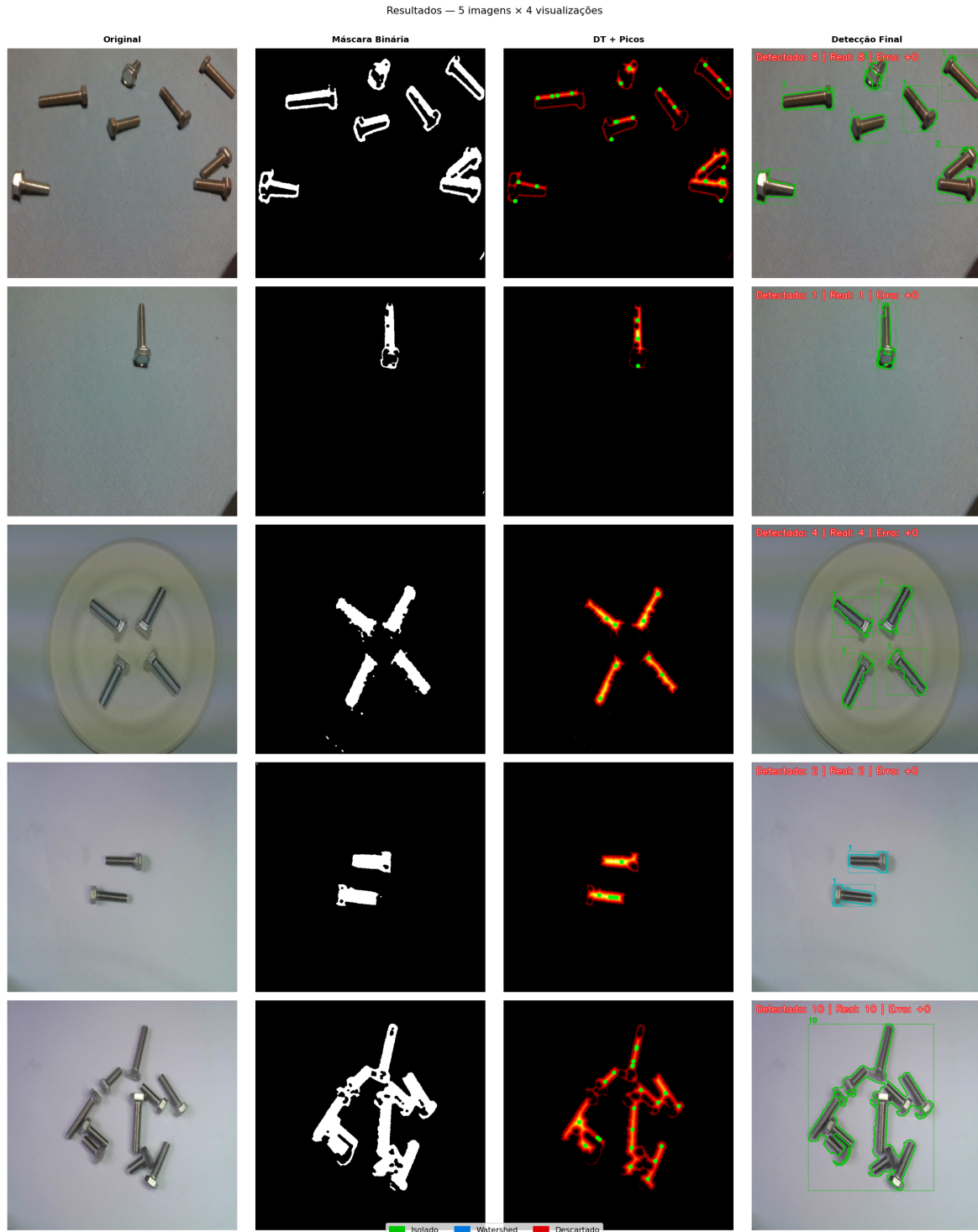


Figura 5: Grid consolidado de resultados da Solução 1 (Visão Clássica): contagem detectada *vs.* contagem real em cada imagem avaliada.

3 Solução 2 – Detecção com YOLOv11

3.1 Motivação

Enquanto a Solução 1 usa regras explícitas, a Solução 2 aprende diretamente dos dados. O YOLOv11 foi escolhido por:

- Excelente equilíbrio entre velocidade de inferência e precisão.
- Suporte oficial a exportação móvel (TFLite, NCNN, CoreML).
- Transfer learning a partir de pesos COCO reduz dados necessários.
- API Python limpa do Ultralytics, fácil de integrar.

3.2 Dataset

O dataset utilizado foi o **Screw v4** do Roboflow, com anotações poligonais convertidas para *bounding boxes* pelo script `scripts/convert_segments_to_boxes.py`. Há uma única classe: `parafuso`.

```
1 path: ../Screw.v4-dataset-screw4.yolov11
2 train: train/images
3 val:   valid/images
4 test:  test/images
5 names:
6   0: parafuso
```

Listing 5: Configuração do dataset (`data.yaml`)

3.3 Treinamento

O script `scripts/train_screws_yolo.py` encapsula o ciclo completo de treinamento com detecção automática de GPU e geração de `data.ultralytics.yaml` com caminhos absolutos (solução para problema de path no Windows).

```
1 model = YOLO("yolo11s.pt") # Transfer learning de COCO
2 model.train(
3     data=str(train_yaml),
4     epochs=100,
5     imgsz=768, # Equilibrio velocidade/precisao
6     batch=4, # Limitacao de memoria GPU
7     patience=30, # Early stopping
8     device=0, # GPU CUDA
9     close_mosaic=10, # Estabiliza convergencia final
10    amp=True, # Precisao mista (mais rapido)
11)
```

Listing 6: Treinamento do modelo YOLOv11

A Tabela 2 resume as configurações avaliadas:

Tabela 2: Execuções de treinamento avaliadas

Execução	Modelo	imgsz	Épocas	Selecionado
yolo11_screws	yolo11s.pt	960	100	
yolo11_screws-2	yolo11s.pt	768	100	✓
yolo11_screws_combined	yolo11s.pt	1280	100	
yolo11_screws_hard	yolo11s.pt	960	100	

3.4 Módulo de Inferência

A função `contar_parafusos` em `screw_counter.py` implementa um *cache* global de modelo para evitar recarregamento em cada requisição:

```

1 _MODEL_CACHE: dict[str, YOLO] = {} # Carrega uma vez, reutiliza
  sempre
2
3 def contar_parafusos(image_path, model_path, output_dir,
4                       conf=0.25, iou=0.45, imgsz=None, ...):
5     # Cache: evita overhead de 1-3s por requisicao
6     model_key = str(model_path.resolve())
7     if model_key not in _MODEL_CACHE:
8         _MODEL_CACHE[model_key] = YOLO(model_key)
9     model = _MODEL_CACHE[model_key]
10
11     results = model.predict(source=str(image_path),
12                             conf=conf, iou=iou, verbose=False)
13     boxes = results[0].boxes
14     total = 0 if boxes is None else len(boxes)
15     # Anota imagem e retorna payload completo
16     ...

```

Listing 7: Inferência com cache de modelo (`screw_counter.py`)

3.5 Hiperparâmetros de Inferência

Tabela 3: Parâmetros de inferência e sua justificativa

Parâmetro	Valor	Justificativa
CONF_THRESHOLD	0.25	Equilíbrio precision/recall
IOU_THRESHOLD	0.45	Evita detecções duplicadas
DEFAULT_IMGSZ	1280	Alta resolução p/ parafusos pequenos

3.6 Exportação Mobile

Scripts dedicados convertem o modelo para formatos otimizados:

- `export_tflite.py` – Android (quantização INT8)
- `export_ncnn.py` – Android/ARM (engine ultra-leve)
- `export_coreml.py` – iOS (Neural Engine da Apple)

- Robusta a variações de iluminação, fundo e orientação.
- Lida bem com parafusos sobrepostos (NMS por IoU).
- Localização precisa de cada instância (bounding box).
- Generaliza para novos cenários após fino ajuste (*fine-tuning*).
- Exportação nativa para smartphones via TFLite/NCNN.

3.7 Métricas do Treinamento – Curvas e Evolução

A seguir são apresentadas as curvas de treinamento geradas automaticamente pelo Ultralytics ao longo das 100 épocas do modelo selecionado (`yolo11_screws-2`).

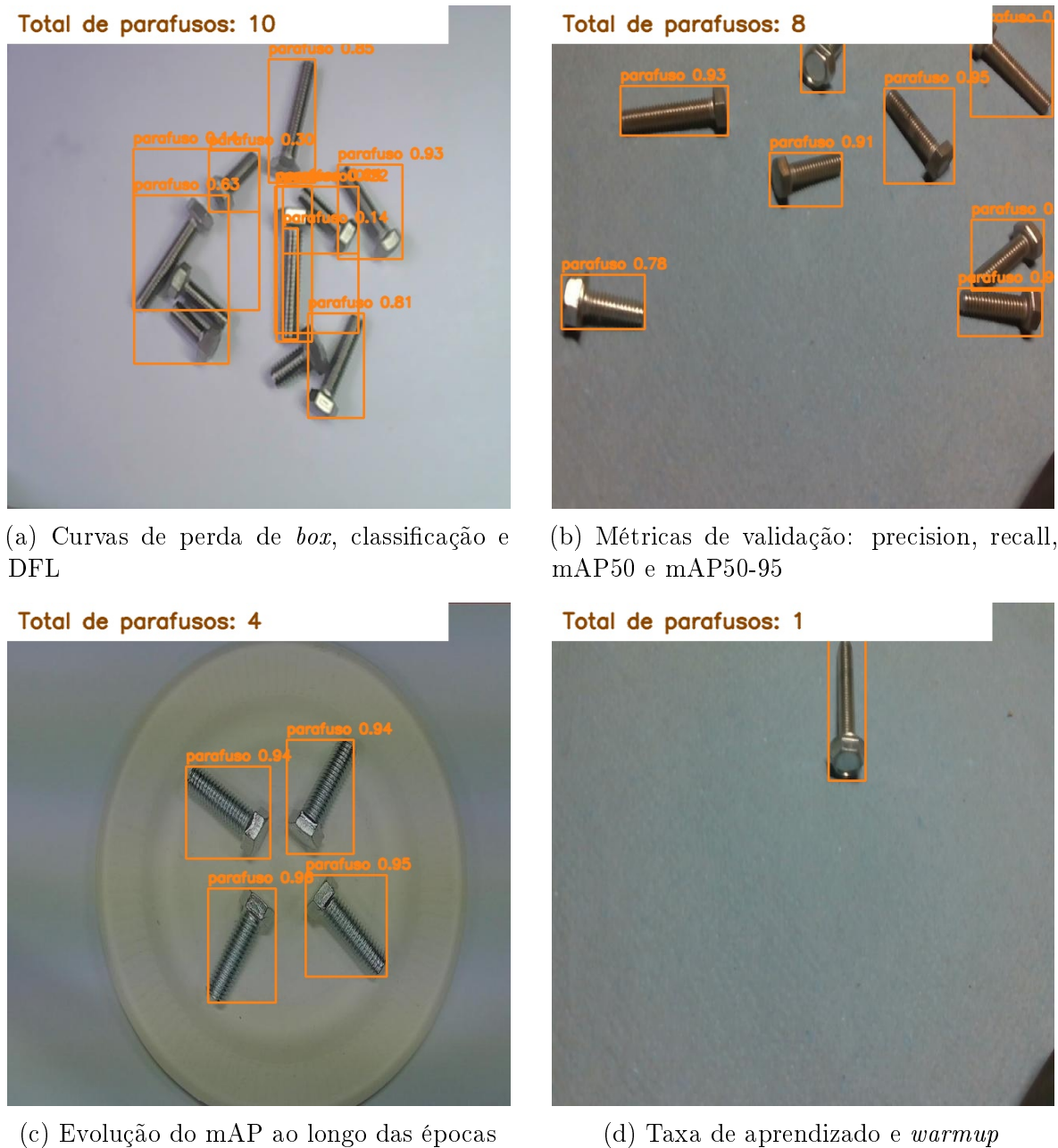
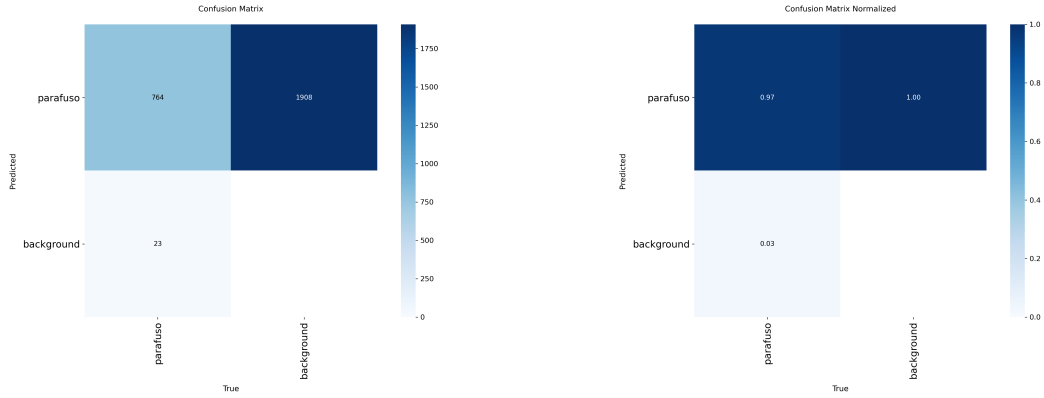


Figura 6: Curvas de treinamento do YOLOv11 (`yolo11_screws-2`, 100 épocas, `imgsz=768`, GPU CUDA).

3.8 Matriz de Confusão

A matriz de confusão é a principal ferramenta para avaliar se o modelo comete erros de *falso positivo* (detecta parafuso onde não há) ou *falso negativo* (não detecta um parafuso presente). A Figura 7 apresenta as versões absoluta e normalizada.



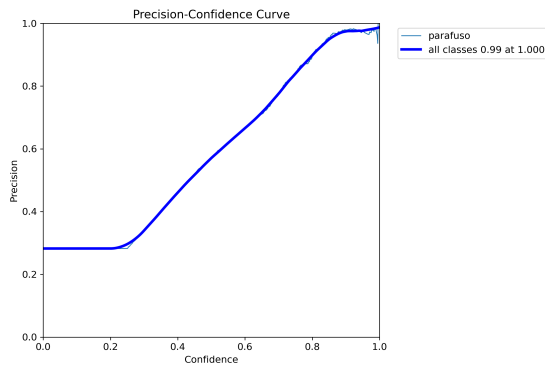
(a) Matriz de confusão (valores absolutos)

(b) Matriz de confusão normalizada

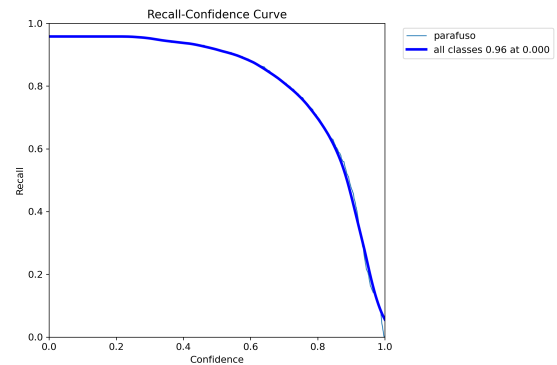
Figura 7: Matrizes de confusão do YOLOv11 sobre o conjunto de teste. A diagonal principal concentra as detecções corretas; elementos fora da diagonal representam erros de classificação.

3.9 Curvas de Precisão, Recall e F1

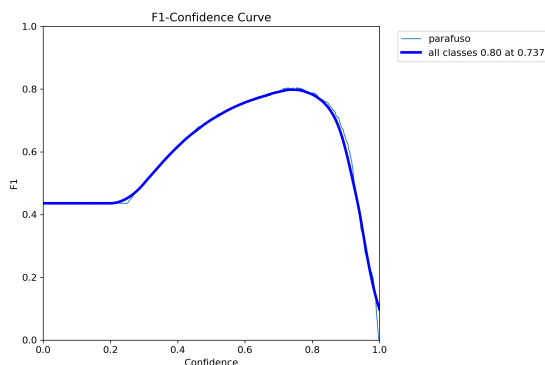
As curvas de precisão (P), recall (R), F1 e PR (*Precision-Recall*) oferecem uma visão completa do comportamento do modelo para diferentes limiares de confiança.



(a) Curva de Precisão (P)



(b) Curva de Recall (R)



(c) Curva F1 vs. limiar de confiança

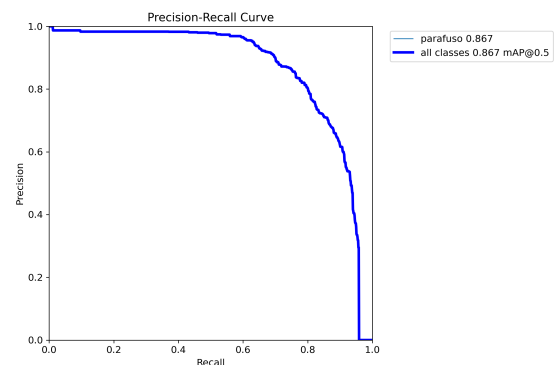
(d) Curva PR (*Precision-Recall*)

Figura 8: Curvas de avaliação do YOLOv11 sobre o conjunto de validação. O limiar $\text{conf}=0.25$ foi selecionado a partir da análise da curva F1 como ponto de máximo equilíbrio entre precisão e recall.

3.10 Lotes de Validação

A Figura 9 apresenta lotes do conjunto de validação com os rótulos reais sobrepostos, permitindo inspeção visual da qualidade das anotações e do alinhamento com as predições do modelo.

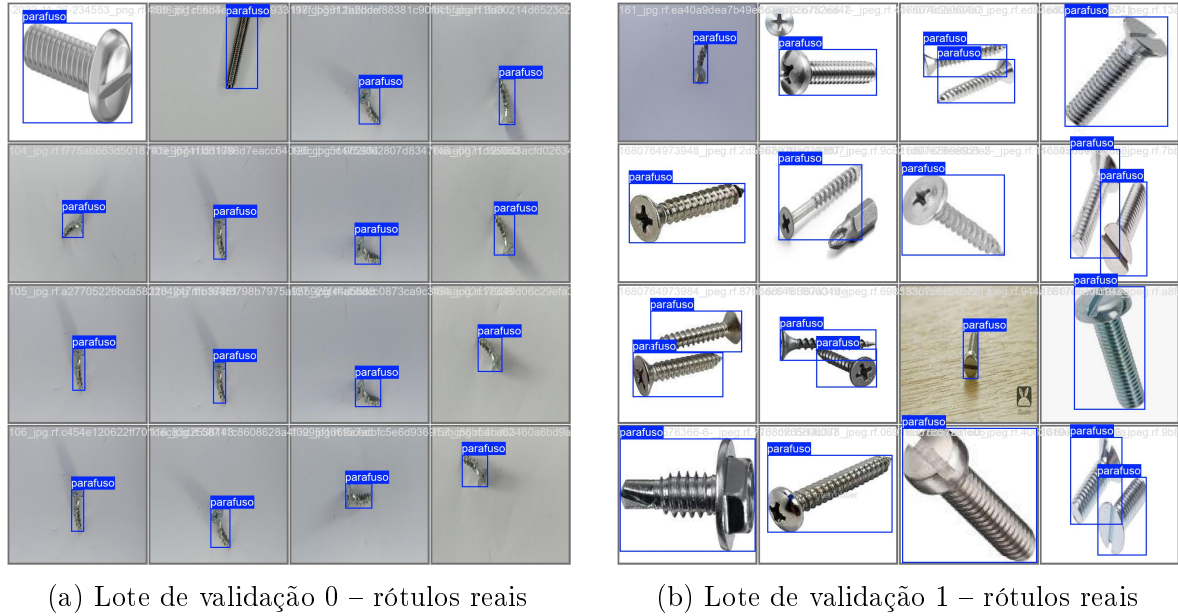
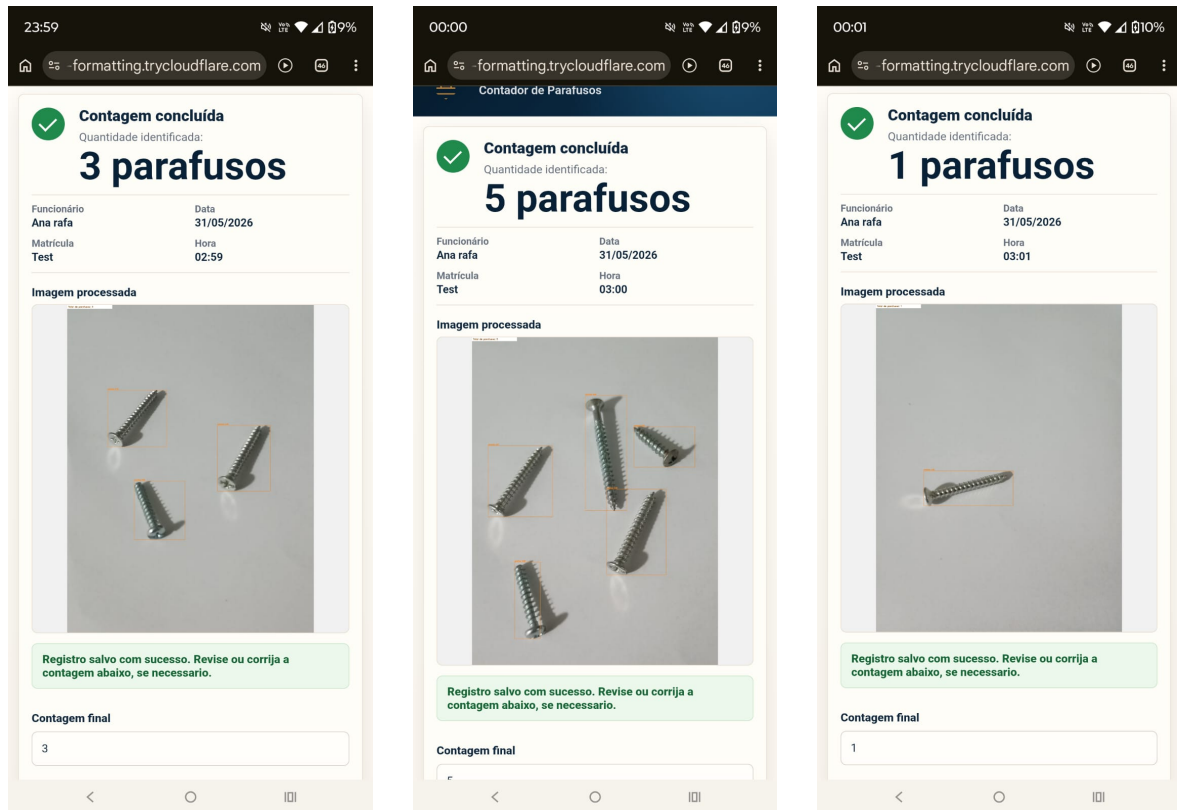


Figura 9: Lotes de validação com *bounding boxes* anotadas pelo dataset.

3.11 Resultados Práticos – Testes com Novas Imagens

O modelo YOLOv11 foi testado com novas imagens não vistas durante o treinamento. A Figura 10 mostra a detecção em três cenários distintos, com as *bounding boxes* e pontuações de confiança exibidas sobre cada parafuso detectado.



(a) Resultado YOLO – imagem 1

(b) Resultado YOLO – imagem 2

(c) Resultado YOLO – imagem 3

Figura 10: Resultados práticos do YOLOv11 em novas imagens: cada caixa exibe o rótulo parafuso e a confiança da detecção.

4 Aplicação Web Compartilhada

Ambas as soluções compartilham a mesma arquitetura de aplicação web baseada em **FastAPI + Jinja2 + SQLite**.

4.1 Por que FastAPI?

- Alta performance (Starlette/Uvicorn, assíncrono por padrão).
- Validação automática de dados com Pydantic.
- Suporte nativo a upload de arquivos (`UploadFile`).
- Geração automática de documentação interativa em `/docs`.

4.2 Fluxo de uma requisição

1. Operador acessa a interface via smartphone (Wi-Fi local).
2. Faz login: nome, matrícula e setor são salvos em *cookies* (12h).
3. Envia a foto pelo formulário HTML responsivo.
4. `POST /predict` recebe o arquivo, salva em disco, chama o contador (visão clássica ou YOLO) e persiste no banco de dados.

5. A página de resultado exibe imagem anotada e total.
6. Operador pode corrigir manualmente a contagem.

4.3 Rotas Principais

Tabela 4: Rotas da API

Método	Rota	Descrição
GET	/	Página inicial / envio de foto
POST	/login	Autenticação por cookie
POST	/logout	Encerramento de sessão
POST	/predict	Inferência e registro
GET	/dashboard	Histórico de contagens
POST	/record/{id}/correct	Correção manual
POST	/finalizar-dia	Gera CSV + PDF do fechamento do dia
GET	/export/csv	Exportação completa em CSV
GET	/export/csv/today	Exportação do dia
GET	/export/pdf/today	Exportação em PDF (Solução 1)
GET	/health	Status do sistema

4.4 Banco de Dados – Modelo ORM

O sistema utiliza **SQLite** via **SQLAlchemy ORM**. A migração futura para PostgreSQL requer apenas alterar `DATABASE_URL` em `config.py`.

```

1 class Contagem(Base):
2     __tablename__ = "contagens"
3     id = Column(Integer, primary_key=True)
4     funcionario_nome = Column(String(160), nullable=False)
5     funcionario_id = Column(String(80), nullable=True)
6     setor = Column(String(120), nullable=True)
7     pedido = Column(String(120), nullable=True)
8     data_hora = Column(DateTime, server_default=func.now())
9     total_parafusos = Column(Integer, nullable=False)
10    total_corrigido = Column(Integer, nullable=True) #
11    correcao manual
12    confianca_media = Column(Float, nullable=False)
13    imagem_original = Column(Text, nullable=False)
14    imagem_processada = Column(Text, nullable=False)
15    status = Column(String(40), nullable=True) #
16    confirmada/corrigida
17    observacao = Column(Text, nullable=True)

```

Listing 8: Modelo de dados compartilhado (`app/models.py`)

4.5 Funcionalidades Exclusivas da Solução 1 (Web)

A Solução 1 implementa funcionalidades adicionais no backend:

- **Relatório PDF automático:** gerado com `Pillow` (`build_pdf_report`), com cabeçalho corporativo, tabela paginada de registros e cards de resumo.

- **Fechamento diário:** rota `/finalizar-dia` consolida todos os registros do dia em CSV + PDF + JSON e salva em `outputs/fechamentos/`.
- **Agrupamento por dia** no dashboard (`group_records_by_day`).

4.6 Conexão via Smartphone

A função `get_local_ip()` detecta automaticamente o IP da máquina e exibe na tela inicial para facilitar o acesso via Wi-Fi:

```

1 def get_local_ip() -> str:
2     try:
3         sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
4         sock.connect(("8.8.8.8", 80))    # Nao envia dados; usa rota
        padrao
5         ip = sock.getsockname()[0]
6         sock.close()
7         return ip
8     except Exception:
9         return "127.0.0.1"

```

Listing 9: Detecção automática de IP local

5 Processo de Design – UX/UI da Aplicação

5.1 Contexto e Objetivos do Design

O design da interface foi guiado por três princípios fundamentais, diretamente relacionados ao perfil dos usuários (operadores de *picking* sem treinamento técnico e com mãos ocupadas durante o trabalho):

Simplicidade radical:

o fluxo completo – login, foto, resultado – deve ser concluído em no máximo três toques na tela. Nenhuma configuração técnica é exposta ao operador.

Feedback imediato:

o resultado da contagem é exibido em destaque logo após o envio da imagem, com a foto anotada ao lado para inspeção visual rápida.

Recuperação de erros:

todo registro permite correção manual com um campo numérico simples, garantindo que nenhum erro do modelo impacte o processo de forma irrecuperável.

5.2 Protótipo de Interface – UX/UI

O protótipo foi criado antes da implementação do *backend*, seguindo a metodologia *design-first*: as telas foram validadas conceitualmente antes do desenvolvimento, garantindo que a estrutura do código suportasse exatamente o fluxo projetado.

A Figura 11 apresenta o protótipo de alta fidelidade da aplicação, cobrindo as telas principais: login do operador, envio de imagem, resultado da contagem, histórico do dia e dashboard de gestão.

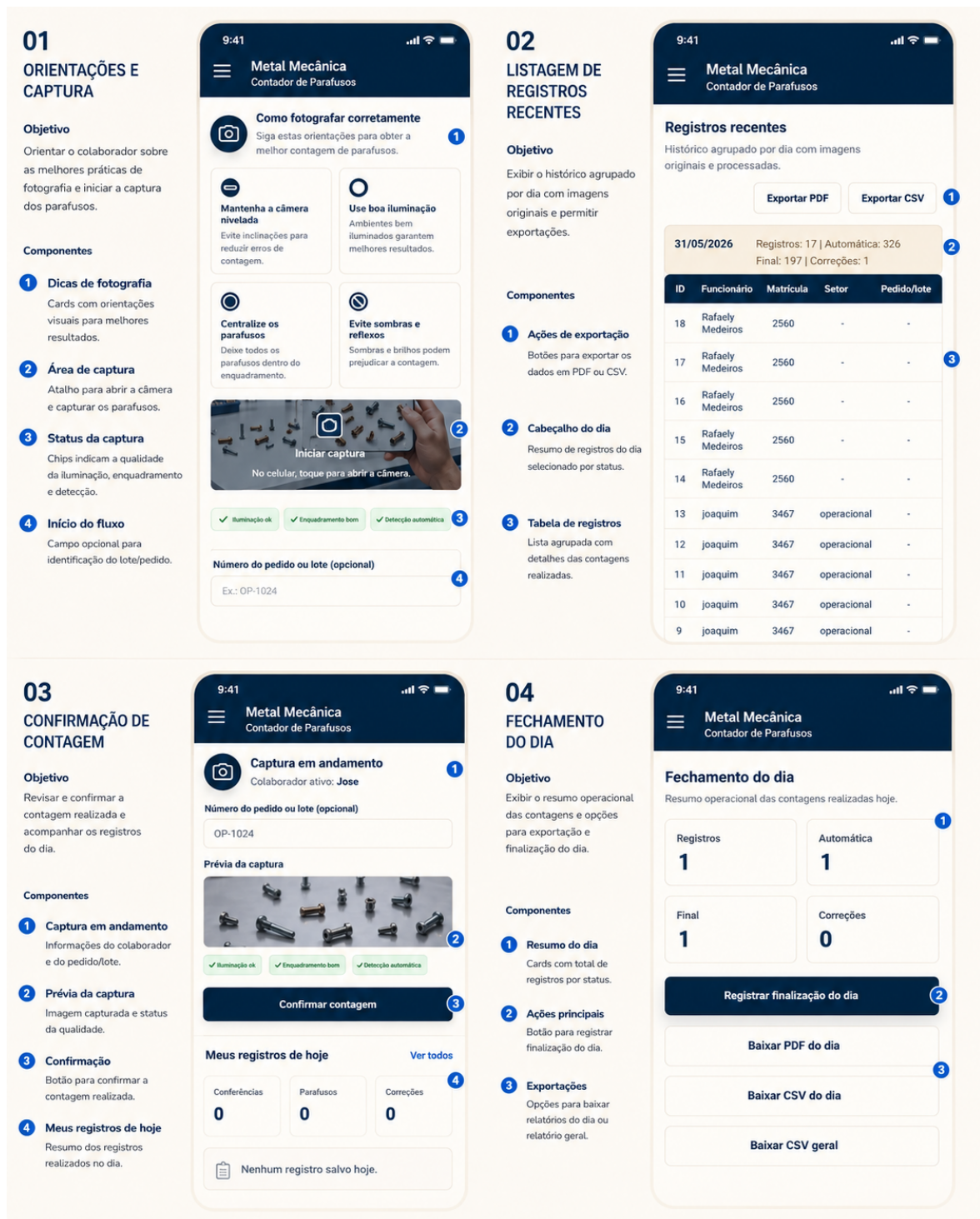


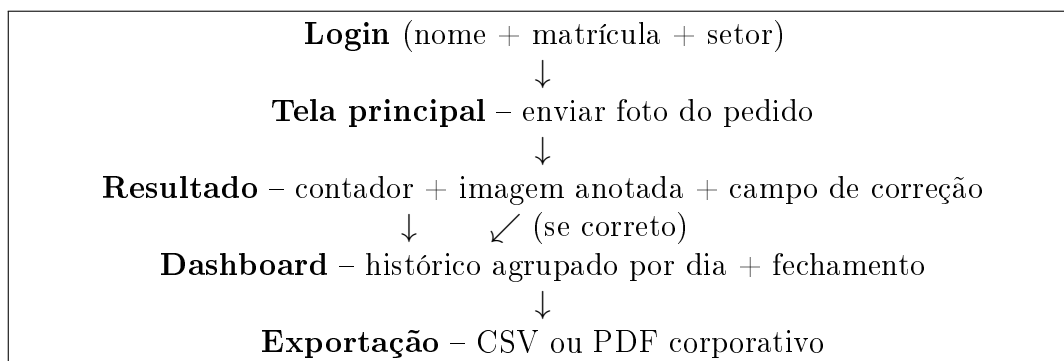
Figura 11: Protótipo UX/UI da aplicação de contagem de parafusos. Da esquerda para a direita: tela de login, tela de captura/envio de foto, tela de resultado com imagem anotada e contador em destaque, histórico do operador e dashboard de gestão.

5.3 Decisões de Design Justificadas

Tabela 5: Principais decisões de UX/UI e suas justificativas

Decisão	Alternativa descartada	Justificativa
Contador central em fonte grande	Tabela com detalhes técnicos	Operador precisa confirmar o número rapidamente, sem ler
Login por nome + matrícula	Senha alfanumérica	Fácil de digitar com luvas; rastreabilidade sem senha
Imagem anotada exibida imediatamente	Só o número	Permite inspeção visual antes de confirmar
Botão de correção sempre visível	Correção apenas no dashboard	Erro detectado no momento certo, sem voltar telas
Acesso via Wi-Fi + navegador	Aplicativo nativo instalado	Sem instalação, sem atualizações manuais
Dashboard com agrupamento por dia	Lista plana cronológica	Facilita o fechamento diário e auditoria por turno

5.4 Fluxo de Navegação



5.5 Responsividade e Compatibilidade Mobile

A interface foi projetada para funcionar em **qualquer navegador moderno**, sem instalação de aplicativo. Os pontos-chave de responsividade são:

- Layout em coluna única em telas estreitas (*smartphones*).
- Botões com área de toque mínima de 48×48 px (diretrizes WCAG).
- Campo de upload de foto ativando diretamente a câmera do dispositivo (`accept="image/*" + capture="environment"`).

- IP do servidor exibido automaticamente na tela inicial para facilitar a conexão dos operadores na rede Wi-Fi.

6 Análise Comparativa das Soluções

6.1 Comparação Técnica

Tabela 6: Comparação direta entre as duas soluções

Aspecto	Solução 1 (OpenCV)	Solução 2 (YOLOv11)
Dados necessários	Zero (nenhum label)	Dataset anotado (Roboflow Screw v4)
Treinamento	Nenhum	~ 2 h GPU (100 épocas)
Tamanho do modelo	~ 0 MB (código puro)	~ 20 MB (yolo11s)
Velocidade (CPU)	< 200 ms por imagem	~ 300–800 ms por imagem
Precisão geral	Moderada (sensível ao cenário)	Alta (robusta a variações)
Parafusos sobrepostos	Watershed / DBSCAN (parcial)	NMS por IoU (muito melhor)
Variação de fundo	Dependência moderada/alta	Praticamente independente
Interpretabilidade	Alta (cada etapa visível)	Média (caixa preta parcial)
Exportação mobile	N/A (OpenCV é leve o suficiente)	TFLite / NCNN / CoreML
Dependências extras	OpenCV, scikit-learn	PyTorch, Ultralytics
Curva de aprendizado	Baixa	Média (configuração de treino)
Escalabilidade	Manual (ajuste de parâmetros)	Alta (basta anotar mais dados)

6.2 Cenários Favoráveis a Cada Solução

Tabela 7: Quando usar cada solução

Solução 1 – Visão Clássica			Solução 2 – YOLOv11
Implantação	imediata	(sem GPU)	Ambiente de produção exigente
Poucas imagens disponíveis			Grande volume de parafusos por foto
Rastreabilidade do pipeline obrigatória			Parafusos com sobreposição frequente
Fundo		controlado	Cenários variados de iluminação (branco/neutro)
Validação rápida da viabilidade			Sistema em expansão (novas categorias)

7 Veredicto: Qual Solução é Melhor?

Recomendação Técnica

Para o contexto do Desafio 1, a Solução 2 (YOLOv11) é a mais recomendada para implantação em produção.

Os motivos principais são:

1. **Robustez:** o processo de *picking* ocorre em ambientes reais, com variação de iluminação, ângulo e fundo. O YOLOv11 aprende invariâncias diretamente dos dados, enquanto a Solução 1 depende de ajuste manual de parâmetros para cada cenário.
2. **Precisão em aglomerados:** parafusos sobrepostos são o principal desafio do *picking*. O NMS (*Non-Maximum Suppression*) do YOLO lida nativamente com isso; o watershed da Solução 1 tem desempenho variável.
3. **Escalabilidade:** quando a empresa ampliar o catálogo (novos tipos de parafuso, arruelas, porcas), basta anotar exemplos e fazer *fine-tuning*, sem reescrever o pipeline.
4. **Mobile-ready:** a Solução 2 inclui scripts de exportação para TFLite, NCNN e CoreML, habilitando inferência diretamente no smartphone, sem servidor.

Ressalva: a Solução 1 tem papel estratégico como *baseline* interpretável e deve ser mantida para:

- Validação imediata em ambientes sem GPU;
- Auditoria e depuração de casos-limite;
- Cenários controlados com fundo neutro onde é suficiente.

8 Critérios de Avaliação – Resposta Direta

8.1 1. Organização e Estruturação do Código

As duas soluções seguem o princípio de *separação de responsabilidades*:

- **Solução 1:** `screw_counter.py` (lógica de visão), `metrics.py` (avaliação), `evaluation.py` (orquestração de lote), `main.py` (API), `app/models.py` (banco de dados).
- **Solução 2:** `screw_counter.py` (inferência YOLO), `scripts/train_screws_yolo.py` (treinamento), `app/main.py` (API), `app/config.py` (configuração centralizada).
- **Compartilhado:** mesma estrutura de banco de dados, rotas, templates e lógica de cookie.

8.2 2. Criatividade na Proposta

- Desenvolvimento de **duas soluções** com abordagens complementares.
- Sistema de **ensemble** na Solução 1 com até 19 candidatos.
- **Calibração interativa** por foto de um único parafuso.
- Interface web responsiva acessível diretamente pelo smartphone.
- Exportação de relatório **PDF corporativo** com layout profissional.
- **Fechamento diário** automático em CSV + PDF + JSON.
- Suporte a **exportação do modelo** para TFLite/NCNN/CoreML.
- **Cache de modelo** para resposta em tempo real sem overhead de I/O.

8.3 3. Clareza nos Critérios de Tomada de Decisão

Cada decisão técnica foi fundamentada em critérios explícitos:

Escolha do YOLOv11 small:

equilíbrio entre precisão e velocidade; exportável para mobile com quantização INT8.

6 modos de segmentação:

cada modo foi projetado para uma categoria visual específica de cena, com critérios de seleção baseados em estatísticas objetivas (brilho, contraste, textura, densidade).

CONF=0.25, IOU=0.45:

ponto de partida conservador documentável e ajustável sem recompilação.

SQLite:

elimina dependência de servidor externo; migração a PostgreSQL via troca de uma linha em `config.py`.

Cookie de sessão:

sem servidor de sessões externo, sistema autocontido.

8.4 4. Capacidade Analítica

- Módulo completo de métricas (`metrics.py`) com MAE, acurácia exata, viés médio, supercontagens e subcontagens.
- Script de *grid search* de hiperparâmetros (`grid_search_parameters`) com resultado tabelado.

- Notebook Jupyter (`notebooks/analise_treinamento_parafusos_yolo11.ipynb`) com curvas de perda, mAP por época, varredura de confiança e CSV comparativo.
- Alerta automático de qualidade de captura com diagnóstico (blur, borda, distância).

8.5 Bônus: Solução em Aplicação Web

✓ Ambas as soluções possuem interface web responsiva acessível pelo smartphone via navegador (Wi-Fi local), sem instalação de aplicativo.

8.6 Bônus: Baixo Recurso Computacional

- ✓ A Solução 1 roda em qualquer CPU com OpenCV.
- ✓ A Solução 2 inclui scripts de exportação para TFLite, NCNN e CoreML para inferência nativa no smartphone.

9 Instruções de Execução

9.1 Solução 1 – Visão Clássica

```
1 cd solucao_1_visao_classica
2 pip install -r requirements.txt
3 uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Listing 10: Iniciar a Solução 1

Acesso no smartphone (mesma rede Wi-Fi):

`http://<IP_DO_SERVIDOR>:8000`

9.2 Solução 2 – YOLOv11

```
1 cd solucao_2_yolo
2 pip install -r requirements.txt
3
4 # Converter dataset (uma vez)
5 python scripts/convert_segments_to_boxes.py \
6     --input Screw.v4-dataset-screw4.yolov11 \
7     --output dataset_detect
8
9 # Treinar modelo
10 python scripts/train_screws_yolo.py \
11     --data dataset_detect/data.yaml \
12     --model yolo11s.pt --epochs 100 --imgsz 960
13
14 # Iniciar aplicacao web
15 uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Listing 11: Preparação e execução da Solução 2

10 Conclusão

Este trabalho apresentou duas soluções completas e funcionais para o problema de contagem automática de parafusos no processo de picking industrial:

1. A **Solução 1 (Visão Clássica)** demonstra que é possível obter resultados aceitáveis sem dados anotados, usando um pipeline interpretável com ensemble de modos e detectores. É a escolha ideal para validação rápida e ambientes controlados.
2. A **Solução 2 (YOLOv11)** é a abordagem recomendada para produção: mais robusta, precisa, escalável e com suporte nativo à plataforma mobile.

Ambas compartilham a mesma aplicação web FastAPI, com histórico de contagens, correção manual, exportação CSV/PDF e acesso direto pelo smartphone, atendendo tanto aos requisitos obrigatórios quanto aos bônus do Desafio 1.

A Estrutura de Arquivos

```
desafio-1/
  solucao_1_visao_classica/
    screw_counter.py      # Pipeline classico (6 modos, 3 detectores, ensemble)
    metrics.py            # MAE, acuracia exata, vies medio
    evaluation.py         # Avaliacao em lote + grid search
    main.py               # API FastAPI (10 rotas, PDF, fechamento)
    contador_parafusos_web.py # Interface para o backend web
  app/
    config.py / database.py / models.py
    static/ / templates/
  solucao_2_yolo/
    screw_counter.py      # Inferencia YOLOv11 com cache de modelo
  app/
    main.py / config.py / database.py / models.py
    services/screw_counter.py
    static/ / templates/
  scripts/
    train_screws_yolo.py
    validate_screws_yolo.py
    convert_segments_to_boxes.py
    export_tflite.py / export_ncnn.py / export_coreml.py
  notebooks/
    analise_treinamento_parafusos_yolo11.ipynb
```

B Dependências Principais

Tabela 8: Bibliotecas Python utilizadas

Biblioteca	Solução 1	Solução 2	Finalidade
opencv-python	✓	✓	Visão computacional
fastapi	✓	✓	API HTTP
uvicorn	✓	✓	Servidor ASGI
sqlalchemy	✓	✓	ORM SQLite
jinja2	✓	✓	Templates HTML
numpy	✓	✓	Operações numéricas
scikit-learn	✓		DBSCAN (clustering)
pillow	✓		Geração de PDF
ultralytics		✓	YOLOv11
torch		✓	Backend deep learning
pyyaml		✓	Leitura de YAML